DNDS5020 - Data Management and Databases
Winter 2026

Project Proposal

Group H

Unifying CEU's Course Information System:
A Student-Centered Database Redesign

Group members

David Diestre Rubio
Teodore Giorgobiani
Thanika Haltrich
Talha Sadiq

January 30, 2026

1 Introduction

1.1 Motivation and Relevance

When looking at the timetable from SITS, David found that he had just one class per week, and although he did come to Vienna on exchange, he does not intend on partying *that* much. This is CEU's course database problem. In David's case, since he is part of the BA department, the courses in the department of Network and Data Science do not show in his timetable. Thanika (also an undergraduate) experienced a similar problem.

Furthermore, a survey conducted in July 2025 found that CEU students were not satisfied with the CEU course selection system [1]; stating that some courses lacked either course descriptions, pre-requisites, or teachers and their experience, making it harder to select courses [1]. Students were also not satisfied with the numerous platforms needed to gather all necessary information (SITS, TimeEdit, Airtable, and Sharepoint), and the occasional contradictions across these platforms [1]. Moreover, students expressed wishes for information to be unified in one platform [1].

1.2 Related work and gap

At Universitat Rovira i Virgili (David's home university), there exists an Intranet where students can find all the mandatory and elective courses they are allowed to take, along with the information about these courses, all in one page [2]. This helps students feel less overwhelmed when selecting courses.

1.3 How we will address this problem

This project aims to address the complaints mentioned in section 1.1, i.e., inconsistent information of courses across platforms and missing key information of some courses. Therefore, what we intend to do is take the existing CEU course database and improve it in a way such that it will be more comprehensive, with the main priority being students. We intend to build a database where students can view:

- What courses they *must* take (mandatories).
- What courses they *can* take (mandatory-electives and electives). The courses being displayed here will have taken pre-requisites into account.
- A function that calculates the credits of students. The main goal is to help students with planning (when they should be taking mandatory-electives to not have problems with graduating).
- A brief summary of the course content, along with the instructor and their experience.

However, in order to do this, we must build a new database entirely. This means we will also take into account information regarding instructors, departments, programs, etc. in our database. The full schema can be seen in section 3.1. That being said, we hope that, in the end, faculty members will also benefit from the new database (in aspects such as easier planning of when a course should be held throughout the term, appropriate times for makeup classes, etc.).

1.4 Findings and Implications

After a brief inspection of the CEU course dataset, we decided on the entities and mapped out the relationships between them. From this inspection, we found that we will not be able to address all the initial issues we spotted, the main one being course prerequisites. Since this column is in natural language (humans typed this in with no strict format), we will not be able to map out **all** courses with their prerequisites (our alternative solution is explained in section 3.3, bullet point 2). However, we still believe that our project will greatly enhance the course selection experience of students, build a better way of displaying student schedules (by displaying courses they are taking in all departments, not just the

department they are in), and improve the university experience overall (students will know early on when and where a course is being held, which will reduce stress at the beginning of the term). Furthermore, we hope the database will be beneficial to faculty members as well.

2 Overview of the data management problem

2.1 What data exists and why it is hard to manage

At CEU, course information is scattered across four platforms: SITS (student records and timetables), TimeEdit (room bookings and scheduling), Airtable (course catalogues in some departments), and Sharepoint (syllabi and course descriptions). Each system was built for a different purpose and operates independently.

The main challenge is heterogeneity. These platforms don't share a common schema: a course code in SITS might follow a different naming convention than in Airtable, making cross-referencing difficult. Because each system operates independently, course information doesn't always appear consistently across platforms. Some courses may be visible in one system but missing from another, forcing students to check multiple sources to get a complete picture.

Access rules create additional problems. As David experienced, SITS filters timetables by departmental affiliation, so a BA student cannot see courses offered by the Department of Network and Data Science, even if those courses are open to them. Students remain unaware of relevant electives simply because the filtering logic is too restrictive.

2.2 Stakeholders and use cases

The primary users would be students who need reliable information to plan their schedules, but advisors, administrators, and faculty would also benefit. Concrete use cases include:

- A student searching for all courses available to them across departments, filtered by schedule and prerequisites.
- An advisor querying which courses satisfy specific degree requirements.
- An administrator identifying scheduling conflicts for courses required by the same cohort.
- A faculty member viewing their consolidated teaching schedule and room assignments.
- A student comparing course descriptions, instructors, and credits from a single source.

2.3 Why a database (and roughly how complex)

A relational database fits this problem because the data is structured around entities with clear relationships: courses connect to instructors, departments, programs, and students. Based on our current design, we estimate around 13 tables will be needed - 5 core entities and 8 junction tables to handle many-to-many relationships.

Entity	**What it represents**
Course	A course with code, department, and exclusion rules
Instructor	A faculty member who teaches courses
Department	An academic unit that offers courses and houses instructors
Program	A degree program with required and elective courses
Student	A student enrolled in a program

Table 1: Draft of the expected core entities

The relationships between these entities require junction tables; some even need multiple depending on the relation: TEACHING_COURSE links instructors to courses; DEPARTMENT_INSTRUCTOR associates instructors with departments; the tables STUDENT_REQUESTED_ENROLLMENT_IN_COURSE (pending requests), STUDENT_ENROLLED_IN_COURSE (enrollment), and STUDENT_PASSED_COURSE (completed courses) are key for different stages of the student-course interaction; and the three program-course tables (PROGRAM_REQUIRED_COURSE, PROGRAM_ELECTIVE_COURSE, PROGRAM_MANDATORY_ELECTIVE_COURSE) capture different types of degree requirements.

3 Sketch of the proposed solution

3.1 ER model

Since this is the proposal phase, our ER model is *keys-only*. This is to validate structure, cardinalities, and constraints, for which we don't need the implementation of non-key attributes (e.g., names, credits, terms, grades, year, etc.). The following ER model schema allows us to communicate the conceptual structure: entities, junction tables, and key-based relations (including the optional self-relations in `Course` for exclusions and hard prerequisites).

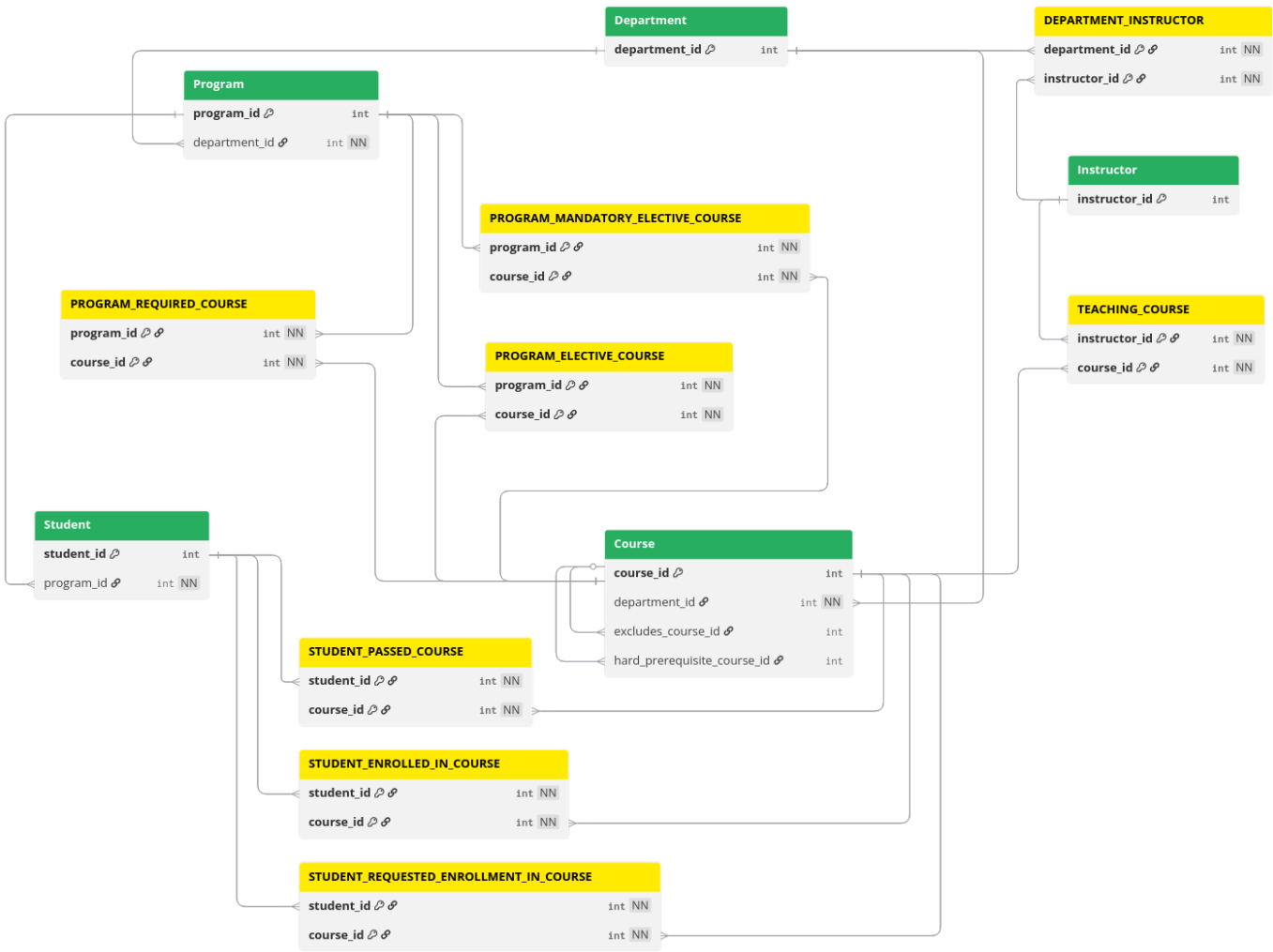

Figure 1: ER model at proposal stage (keys-only). Entities are shown in green, and junction tables in yellow. Image created using dbdiagram.io [3]

3.2 Why PostgreSQL

We will implement the solution in **PostgreSQL** because it handles complex joins efficiently and supports constraints for data integrity.

For this project, no special databases or extensions are required, since the data doesn't change at high frequency and will mostly consist of queryable information rather than large texts. The only longer free-text field we anticipate is a `prerequisites_text` attribute for the courses that have specific requirements other than / on top of specific subjects.

3.3 Some query ideas

To demonstrate how this database model can provide useful functionalities not available with the actual implementation, we outline three example query ideas:

- **Course planning for next year:** A student can request a list of courses they can take next year based on their program, year of study, and previously passed courses. With attributes such as "available from year n" and "min ECTS required" on the program-course junction tables, the system can hide electives for first-year students and display advanced courses only when appropriate.
- **Course inscription with minimal instructor workload:** Courses that have other courses as their only prerequisites will have a course-course self-relation (via `hard_prerequisite_course_id`). The system can automatically approve requests (or display them as ready for approval) when the student has passed those prerequisite courses, and otherwise still allow a request but mark it for instructor review (e.g., if the student claims equivalent external preparation). This way, instructors only intervene in exceptional cases or when course prerequisites are specified in more complex text fields (which would be shown in the `prerequisites_text` course attribute).
- **Credit-based elective capacity:** By adding course credits and basic term/year attributes, the system can compute the credits coming from mandatory courses in a given term/year and show students how many (mandatory-)electives they can add. This supports workload planning and helps students decide between mandatory-electives and free-choice electives.

4 Timeline and tasks (GANTT chart)

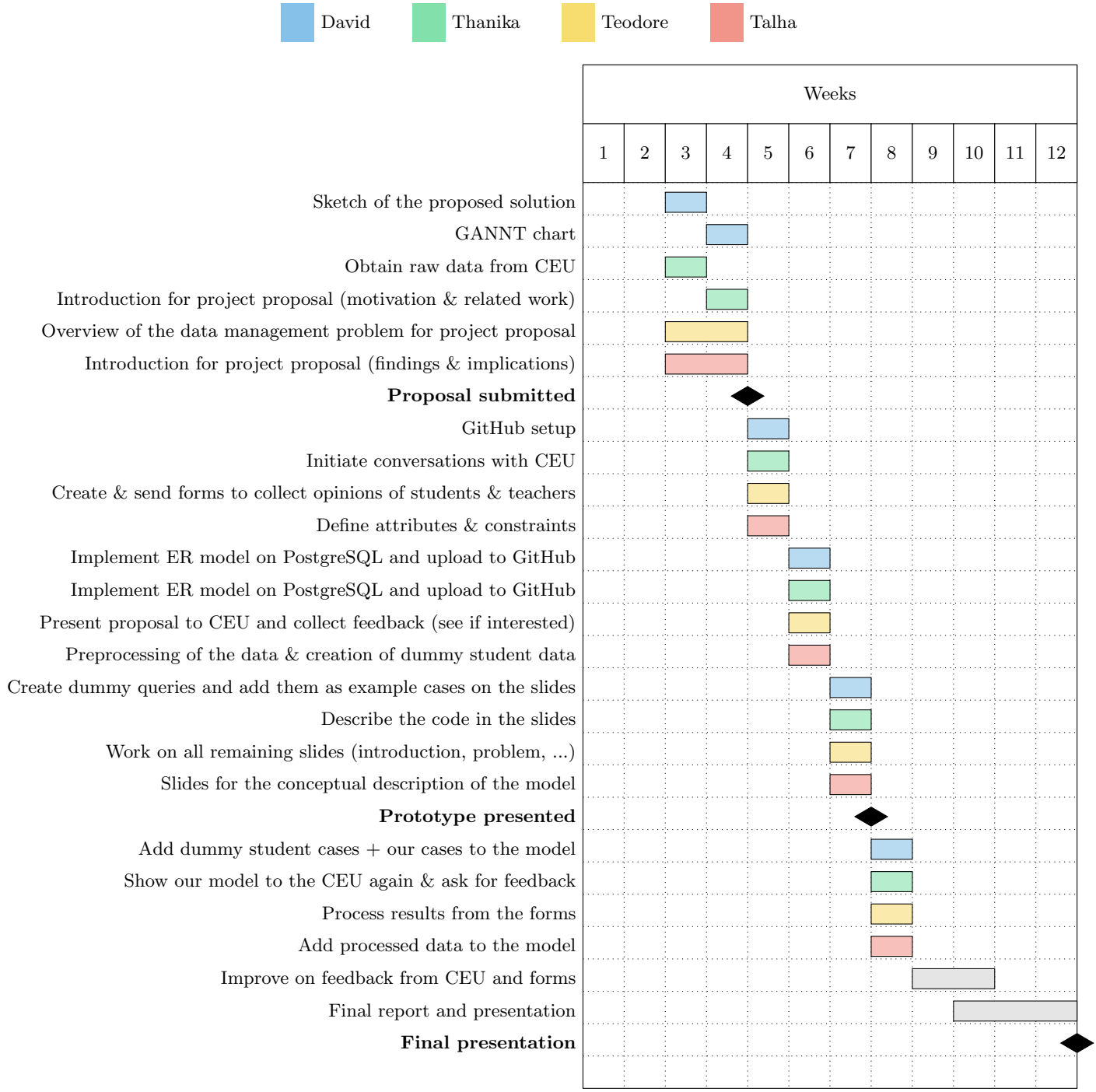

Figure 2: Gantt chart

References

[1] E. Lvov and T. Haltrich. *CEU Students' experience with course selection process*. Unpublished raw data, Central European University. 2025.

[2] Universitat Rovira i Virgili. *Universitat Rovira i Virgili Intranet*. Access restricted; available only with David's URV login credentials. 2026. URL: https://intranet.urv.cat/group/intranet/.

[3] Holistics Software. *dbdiagram.io*. Used to create the ER model representation in this project proposal. 2026. URL: https://dbdiagram.io/.